Notes — Atelier 4: Service layer, IoC and DI

A. Injection modes compared

Criterion	Constructor	Field (`@Autowired`)
`final` field possible	Yes	No
Dependency visible in the class signature	Yes	No (hidden in a private field)
Usable with `new` outside the container without a `null` dependency	The dependency must be provided: the object is always complete	Yes, but the dependency stays `null` (NullPointerException)
Flagged by SonarQube for IDE	No	Yes (rule S6813, to confirm in your IDE)

B. Services

Service	Injected dependencies	Mode	Justification
AgenceServiceImpl	IAgenceRepository	Constructor (`@RequiredArgsConstructor`, `final` field)	Required dependency typed by an interface, object never incomplete, no new.
EmployeServiceImpl	IEmployeRepository	Constructor (`@RequiredArgsConstructor`, `final` field)	Required dependency typed by an interface, object never incomplete, no new.
VehiculeServiceImpl	IVehiculeRepository	Constructor (`@RequiredArgsConstructor`, `final` field)	Required dependency typed by an interface, object never incomplete, no new.
EquipementServiceImpl	IEquipementRepository	Constructor (`@RequiredArgsConstructor`, `final` field)	Required dependency typed by an interface, object never incomplete, no new.
ClientServiceImpl	IClientRepository	Constructor (`@RequiredArgsConstructor`, `final` field)	Required dependency typed by an interface, object never incomplete, no new.
ReservationServiceImpl	IReservationRepository	Constructor (`@RequiredArgsConstructor`, `final` field)	Required dependency typed by an interface, object never incomplete, no new.
ContratServiceImpl	IContratRepository	Constructor (`@RequiredArgsConstructor`, `final` field)	Required dependency typed by an interface, object never incomplete, no new.
PaiementServiceImpl	IPaiementRepository	Constructor (`@RequiredArgsConstructor`, `final` field)	Read-only: a payment is created, modified and deleted through its contract.
MaintenanceServiceImpl	IMaintenanceRepository	Constructor (`@RequiredArgsConstructor`, `final` field)	Required dependency typed by an interface, object never incomplete, no new.

Each service is a stateless singleton: it has no modifiable attribute. The business rules (identifier refused on creation, required fields, negative values, date order) throw an `IllegalArgumentException`; a missing resource throws a `ResourceNotFoundException`.

C. Container error messages

1. **Message 1 —** `NoSuchBeanDefinitionException` (no `IContratService` bean): `ContratServiceImpl` is not annotated with `@Service` (or another stereotype annotation), or it sits in a package outside `tn.esprit.autoloc`, so it is never scanned. Fix: add `@Service` and check the package.

2. **Message 2 —** `NoUniqueBeanDefinitionException` (2 beans: `emailNotificateur`, `smsNotificateur`): (a) `@Primary` **on the class** of the bean to use by default; (b) `@Qualifier("emailNotificateur")` **on the constructor parameter** of the controller, for an explicit choice.
3. **Message 3 — circular dependency** (`clientServiceImpl` ↔ `reservationServiceImpl`): with constructor injection, each bean must exist fully built before the other can be created, so neither can be created first. Design option: extract the shared responsibility into a third service that both depend on, or let one of them go through the other's repository.

D. Experiments and SonarQube for IDE findings

Experiment 1 — field injection (step 2)

In `ContratServiceImpl`, `@RequiredArgsConstructor` and the `final` field were temporarily replaced with:

```
@Autowired
private IContratRepository contratRepository;
```

The application starts normally: field injection works, because Spring sets the field after the object is constructed. SonarQube for IDE still flags the field with the rule "Field dependency injection should be avoided" (S6813; depending on the version of the extension, the older rule S3306 may also appear). Reasons: the field cannot be `final`, the dependency does not appear in the class signature, and the object can be created without its dependency (`null`) outside the container. The code was then restored (constructor and `final` field).

Experiment 2 — generic exception (step 4)

In `ContratServiceImpl.findById`, `ResourceNotFoundException` was temporarily replaced with `RuntimeException`. SonarQube for IDE reports rule S112: a generic exception should never be thrown, because the caller cannot tell it apart from other errors. The business exception `ResourceNotFoundException` makes the error explicit and lets it be translated into a 404 (Atelier 5). The code was then restored.

Findings table

Finding	Rule / explanation	Fix applied
Field injection (`@Autowired` on a field)	S6813: prefer constructor injection (`final` field, visible dependency, object always complete)	Back to `@RequiredArgsConstructor` with a `final` field
`RuntimeException` thrown instead of a business exception	S112: do not throw a generic exception	Back to `ResourceNotFoundException`
`LocalDate.now()` without a time zone (`CascadeDemoRunner`)	The result depends on the machine's time zone; a `ZoneId` or a `Clock` must be passed (rule key to read in the panel)	`LocalDate.now(ZoneId.systemDefault())`
Return value of `size()` ignored (`CascadeDemoRunner`)	A call with no side effect whose result is ignored is pointless (probably S2201, to confirm in the panel)	Result stored in a variable and printed in the log

The last two rows come from the analysis of the whole project (Atelier 3); the analysis of the `service` and `exception` packages revealed no other finding.

E. Comprehension questions

1. **Where is the** `new` **of** `ContratServiceImpl`**?** In the Spring container: at startup, the scan detects `@Service`, then the container calls the constructor and provides the repository proxy.
2. **Why depend on** `IContratService`**?** To depend only on an abstraction: the implementation can change without touching the controller, and the layers are respected (the controller knows neither the repository nor the implementation).

3. **Why must a singleton stay stateless?** It is shared by all threads. Example of a bug: an attribute `contratCourant` set by one request and read by another returns another user's data.
4. **Why does `update` load, then copy?** `save` with the received object would do a `merge` that overwrites every column missing from the JSON (they would become `null`) and, with `orphanRemoval`, could empty the collection of payments.
5. **Differences.** `@Component` goes on a class you write; `@Bean` goes on a method, for a class you cannot annotate (library, JDK). `@Primary` marks the default candidate, placed on the class; `@Qualifier` makes an explicit choice, placed at the injection point. `IPaiementService` only exposes reads because a payment exists only through its contract (composition): creating or removing a payment goes through the `Contrat`.